



11 MÓDULOS · 24 CHECKPOINTS · EJERCICIOS GUIADOS

Claude Code, de la sesión al script.

Un agente que vive en tu terminal, lee tu proyecto y ejecuta comandos. Este curso te lleva de la primera sesión a tenerlo trabajando solo: con memoria, con límites que decides tú, y con lo que repites convertido en un comando.

No hace falta que vivas en la terminal. Cada comando viene explicado antes de que lo escribas, y cada ejercicio te dice qué deberías ver en pantalla para saber que salió bien. Lo que marcas aquí se guarda en este navegador; lo que de verdad cuenta lo confirma `./verificar.sh` contra tu propia instalación.

11

MÓDULOS

17

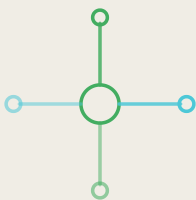
CHECKPOINTS AUTO

7

CHECKPOINTS MANUALES

12

EJERCICIOS GUIADOS



MÓDULO 0

Arranque

Qué es Claude Code, cómo comprobar que quedó bien instalado y cómo es una sesión.

Un agente que vive en la terminal

LA CONFIANZA ES EL RIESGO, NO LA INCOMPETENCIA

El fallo que más caro sale con estas herramientas no es que se equivoquen: es que se equivocan con el mismo tono con el que aciertan. Un compilador que falla da un error; un agente que falla da una respuesta bien redactada.

Eso desactiva la alarma que sí tienes con otras herramientas, y explica por qué las capas de este curso —verificación, hooks, subagente independiente— no son adornos: son el sustituto de esa alarma.

La costumbre práctica que más protege: **pide siempre que compruebe, y comprueba tú que comprobó**. «Las pruebas pasan» es una afirmación que se verifica en dos segundos.

Lo que sigue son once módulos, y el hilo que los une es uno solo: pasar de pedirle cosas a montarle un sitio donde trabajar. Reglas para que sepa dónde está, permisos para que pueda actuar, verificación para que no se declare terminado antes de tiempo, y automatización para que no dependa de que tú te acuerdes.

Una advertencia sobre el orden de lectura: los módulos del 3 al 8 —comandos, skills, subagentes, hooks, MCP, plugins— son capacidades que se añaden cuando aparece la necesidad, no una lista que haya que completar.

Si intentas montarlo todo antes de trabajar, acabas con una configuración elaborada que no usas y que además te estorba. Haz los tres primeros, trabaja una semana, y vuelve al que resuelva lo que te haya molestado.

«Arregla este fallo»	Reproducirlo ejecutando el proyecto, y verificar el arreglo
«Añade una prueba»	Correrla y comprobar que falla antes y pasa después
«¿Por qué cambió esto?»	Buscar en el historial de git y leer los commits
«Actualiza esta dependencia»	Instalarla y ver si algo se rompió

La columna derecha es la que justifica todo el curso. Sin ella, cualquier chat sirve; con ella, el ciclo de escribir-comprobar-corregir ocurre sin que tú seas el mensajero entre las dos mitades.

Sobre para quién es este curso: para quien ya programa. Estas herramientas aceleran a quien sabe leer el resultado y multiplican los errores de quien no — con una seguridad que hace difícil detectarlos. No es una advertencia moral, es una descripción de cómo funciona.

Y sobre qué esperar al final: once módulos, veinticuatro checkpoints y un proyecto tuyo configurado. No vas a salir sabiéndolo todo; vas a salir con las cinco capas montadas —memoria, permisos, comandos, verificación y automatización— y con criterio para decidir cuándo añadir la sexta.

Vale la pena situar de qué tamaño es el cambio. No estás aprendiendo una herramienta más: estás cambiando quién ejecuta. Hasta ahora tú escribías y la máquina obedecía instrucciones exactas. Ahora describes un resultado y algo intermedio decide los pasos.

Eso trae una habilidad nueva que no tenías que ejercitar antes: **saber cuándo desconfiar**. Un compilador que se equivoca da un error; un agente que se equivoca da una respuesta con la misma seguridad que cuando acierta. Buena parte de este curso es montar comprobaciones para no depender de esa seguridad.

Una conversación con el agente, desde que escribes `claude` hasta que sales. Al salir se pierde, salvo lo que hayas dejado escrito en archivos.

contexto

Todo lo que el agente tiene delante al responder: tu petición, los archivos que leyó, las salidas de los comandos que ejecutó y las reglas del proyecto. Es limitado.

herramienta

Cada cosa que el agente sabe hacer además de escribir: leer un archivo, ejecutar un comando, buscar en la web. Los permisos deciden cuáles puede usar sin preguntarte.

CLAUDE.md

El archivo de reglas del proyecto. Se lee solo al empezar cada sesión.

hook

Un comando tuyo que se ejecuta automáticamente antes o después de que el agente use una herramienta. Es código, no una petición.

MCP

Un protocolo para conectar el agente a sistemas externos —una base de datos, un gestor de incidencias— con herramientas que aparecen como propias.

Antes de nada, la pregunta que decide si este curso te sirve: **¿pasas más tiempo en el editor o en la terminal?** Si es lo primero, Claude Code te va a parecer incómodo las primeras horas. Si es lo segundo, va a encajar en lo que ya haces desde el primer día.

NO ES UN CHAT QUE ADEMÁS ESCRIBE CÓDIGO

La confusión más común es tratarlo como un asistente de conversación al que le pides fragmentos. Si lo usas así, es peor que un chat en el navegador — tiene menos interfaz y ninguna ventaja.

La diferencia está en que **puede actuar y comprobar**: lee tus archivos, ejecuta tus pruebas, ve el error real y vuelve a intentarlo. Ese ciclo cerrado es todo lo que ofrece de más, y es mucho.

La consecuencia práctica: en vez de «escribeme una función que valide correos», pídele «añade validación de correo al formulario de registro y comprueba que las pruebas siguen pasando». Lo segundo aprovecha lo que sabe hacer; lo primero no.

Y una advertencia sobre el ritmo del curso. Son once módulos y no hay que hacerlos seguidos. Los tres primeros —arranque, memoria y permisos— son los que cambian el día a día. Del cuatro en adelante son capacidades que se añaden cuando las necesitas, y si intentas configurarlo todo antes de trabajar acabarás con un montaje que no usas.

Claude Code no es un editor ni una extensión: es un programa de terminal al que le hablas en español y que **trabaja sobre la carpeta donde lo abriste**. Lee tus archivos, los modifica, ejecuta comandos y lee lo que devuelven.

Esa última parte es la que lo diferencia de un chat. Un chat te da una respuesta y te toca a ti probarla. Un agente en la terminal **puede ejecutar la prueba, ver que falla y volver a intentarlo** sin pasar por ti.

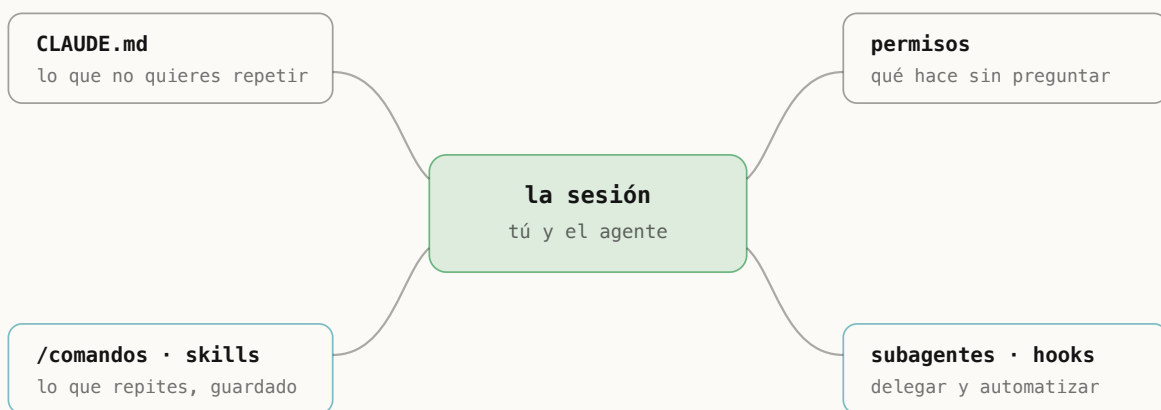
EL BUCLE QUE LO DEFINE

Le pides algo. Lee lo que necesita, propone cambios, ejecuta lo que haga falta para comprobarlos, y mira el resultado. Si salió mal, corrige y repite.

Tú entras en dos momentos: cuando quiere hacer algo que no le has autorizado, y al final, cuando revisas lo que hizo. Todo el curso trata de **afinar esos dos momentos**: que te pregunte lo justo, y que lo que te entregue se pueda revisar.

Y conviene decir lo que **no** es:

- **No es autónomo por defecto.** Pide permiso antes de tocar cosas que no le autorizaste. Ese comportamiento se configura, y es el módulo 2.
- **No recuerda entre sesiones.** Cada vez empieza de cero, salvo lo que le dejes escrito en un archivo. Eso es el módulo 1.
- **No es solo para programar.** Cualquier cosa que se haga con archivos y comandos le sirve: renombrar cien archivos, revisar un CSV, preparar un despliegue.



En el centro, tú y el agente. Todo lo demás del curso son capas que hacen esa conversación más corta, más segura y más repetible.

Comprobar que quedó bien instalado

Y si algo sigue sin cuadrar después de comprobar las tres cosas, el propio `claude doctor` suele decir qué falta. Es la herramienta de diagnóstico y casi nadie la usa dos veces.

Si trabajas en una empresa con restricciones, este es el momento de comprobarlas: hay configuraciones que se imponen desde la organización y pueden limitar qué modelos o qué funciones tienes disponibles. Descubrirlo ahora ahorra confusión en los módulos de permisos y MCP.

Y si la instalación la gestiona otro equipo, pídeles la versión: buena parte de lo que este curso describe necesita una razonablemente reciente.

Con la instalación comprobada tienes lo mínimo para empezar. Todo lo demás del curso —reglas, permisos, comandos— se monta sobre un proyecto tuyo, así que conviene tener uno a mano antes de seguir. No hace falta que sea grande; hace falta que sea real.

Si algo de este módulo falla y no sabes por dónde seguir, el orden de diagnóstico es siempre el mismo: ¿existe el comando?, ¿responde?, ¿está autenticado? Cada pregunta tiene un arreglo distinto, y saltarse el orden hace perder tiempo.

El verificador está montado con esa misma secuencia, así que su primer fallo suele ser el que hay que arreglar — los siguientes se caen solos cuando resuelves ese.

Un apunte sobre lo que este curso no cubre y conviene saber que existe: hay una extensión para trabajar desde el editor y una versión que corre en el navegador. Son la misma herramienta con otra piel.

El curso se hace entero en la terminal a propósito: es donde están todas las capacidades y donde lo que aprendes se traslada a las otras formas. Al revés no funciona igual.

Merece la pena entender la diferencia entre las dos formas de instalarlo, porque explica casi todos los problemas raros. El instalador nativo deja un binario que se actualiza

solo; la instalación por npm depende de tu versión de Node y se actualiza cuando tú lo pidas.

Ninguna es mejor. Lo que da problemas es tener las dos: la terminal ejecuta una, tú actualizas la otra, y el comportamiento no coincide con lo que lees en ningún sitio.

QUÉ MIRA EXACTAMENTE EL VERIFICADOR DEL CURSO

Los checkpoints de este módulo no te preguntan si instalaste: lo comprueban ejecutando lo mismo que ejecutarías tú.

Buscan el comando en tu `PATH`, le piden la versión, y comprueban que la autenticación esté resuelta. Por eso un checkpoint automático no se puede marcar a mano — no hay nada que marcar: o el comando responde o no.

Cuando uno falle, léelo con cuidado antes de tocar nada: el mensaje distingue «no está instalado» de «está pero no responde» de «responde pero no está autenticado», y cada uno se arregla de forma distinta.

Una cosa más sobre autenticación: hay varias formas y conviven mal. Si tienes una variable de entorno con una clave puesta de hace meses, va a ganar sobre la sesión que inicies ahora, y el resultado es que trabajas contra una cuenta distinta a la que crees. Si algo no cuadra, esa variable es el primer sitio donde mirar.

Una cosa que conviene mirar en la salida de `claude doctor` aunque no la entiendas del todo: **cómo se instaló**. Si dice que fue por npm y tú recuerdas haber usado el instalador —o al revés— tienes dos, y ese es el origen de los comportamientos que después no se explican.

Y sobre las actualizaciones: conviene tenerlas automáticas. Estas herramientas cambian rápido, y una versión de hace tres meses puede no tener funciones que la documentación da por hechas. Si algo del curso no te aparece, actualizar es lo primero que hay que probar.

Un apunte sobre el modelo, porque es la pregunta que sale enseguida: no hace falta elegirlo. Claude Code trae uno por defecto que es el adecuado para casi todo, y se cambia con `/model` si algún día lo necesitas. Empezar tocando eso es optimizar antes de tener un problema.

POR QUÉ SE COMPRUEBA ANTES DE PEDIR NADA

Tres cuartas partes de los problemas del primer día no son del agente: son de instalación. Una versión vieja que no tiene los comandos que espera el curso, una autenticación a medias, o una instalación por un gestor de paquetes que se pisa con otra.

`claude doctor` existe justamente para eso y casi nadie lo conoce. Te dice qué versión corre, cómo se instaló y si las actualizaciones automáticas funcionan — las tres cosas que después explican comportamientos raros.

Dos minutos aquí ahorran una tarde de «a mí no me funciona igual que en el vídeo».

DOS INSTALACIONES A LA VEZ

El problema más difícil de diagnosticar de este módulo: tener Claude Code instalado por dos vías —el instalador nativo y npm, por ejemplo— y que la terminal ejecute una mientras tú actualizas la otra.

El síntoma es que `claude --version` no cambia después de actualizar, o que una función que la documentación describe no aparece. Se comprueba con `which -a claude`: si devuelve más de una ruta, tienes dos, y conviene dejar una sola.

Antes de pedirle nada conviene confirmar tres cosas: que el programa está, que su instalación está sana, y que sabe quién eres. Las tres se comprueban desde la terminal en menos de un minuto.

El primero es el comando más útil del curso y casi nadie lo conoce: `claude doctor` revisa la instalación y te dice qué versión corre, cómo se instaló y si las actualizaciones automáticas funcionan.

EJERCICIO 0.1 · 6 minutos · revisar tu instalación

OBJETIVO

Confirmar que Claude Code está instalado, sano y autenticado, sin abrir todavía una sesión.

ANTES DE EMPEZAR

Necesitas una terminal. Si el comando `claude` no existe todavía, instálalo desde claude.com/code (<https://claude.com/code>) y reabre la terminal.

PASOS

1. Descarga el kit del curso, que trae el verificador:

● TERMINAL

```
$ git clone https://github.com/gonzalezulises/curso-claude-code
$ cd curso-claude-code
```

2. Pregunta qué versión tienes:

● TERMINAL

```
$ claude --version
```

3. Pídele que se revise a sí mismo:

● TERMINAL

```
$ claude doctor
```

4. Deja que el verificador confirme los tres puntos de este módulo:

● TERMINAL

```
$ ./verificar.sh -m 0
```

QUÉ DEBERÍAS VER

`claude --version` devuelve algo como `2.1.221 (Claude Code)`. `claude doctor` muestra varias líneas: la versión que corre, la plataforma, el método de instalación y el estado de las actualizaciones. El verificador te confirma los checkpoints 0.1 a 0.3 con la versión y la ruta exactas.

SI ALGO FALLA

command not found: claude → o no está instalado, o la terminal se abrió antes de instalarlo. Cierra la ventana, abre otra y repite.

El verificador dice que no detecta autenticación → abre `claude` una vez y sigue el inicio de sesión que te propone. Si usas una clave de API, tiene que estar en la variable `ANTHROPIC_API_KEY` de esa misma terminal.

doctor menciona una versión distinta a --version → suele haber dos instalaciones conviviendo. `claude doctor` te dice la ruta de la que manda.

Claude Code instalado y responde AUTO

La instalación pasa el chequeo de salud AUTO

Tienes una vía de autenticación configurada AUTO

Tu primera sesión

Una última costumbre para el primer día: cuando el resultado no sea el que esperabas, **no reformules la petición desde cero**. Dile qué está mal con lo que hizo. Tiene delante todo el contexto de lo que acaba de hacer, y corregir sobre eso es más barato y más preciso que empezar otra vez.

Al terminar esta sesión ya viste las tres cosas que separan esto de un chat: leyó archivos por su cuenta, ejecutó algo, y comprobó el resultado. Los diez módulos siguientes son formas de hacer que esas tres cosas ocurran mejor y sin que tengas que pedir las cada vez.

Una nota sobre el coste, porque es la duda que frena a mucha gente: cada sesión consume según cuánto lea y cuánto escriba. Una sesión de trabajo normal es barata; la que se descontrola es la que lleva dos horas abierta acumulando contexto.

La costumbre de una sesión por tarea, además de mejorar las respuestas, es lo que mantiene el gasto predecible.

Sobre el modo de planificación, que aparece pronto y confunde: hay una forma de pedirle que **proponga antes de tocar nada**. Te devuelve un plan, tú lo apruebas o lo corriges, y solo entonces actúa.

Merece la pena cogerle el gusto para lo que toque varios archivos. El coste de corregir un plan de cinco líneas es mucho menor que el de revisar quince archivos ya modificados, y es la forma más barata de descubrir que entendiste una cosa y él otra.

Cuando salgas de la sesión, conviene saber qué se pierde y qué no. **Se pierde la conversación**: lo que hablasteis, lo que leyó, lo que dedujo. **No se pierde lo que quedó escrito en archivos**: el código, las notas, las reglas.

De ahí la costumbre que sostiene todo lo demás: si algo importa para mañana, tiene que acabar en un archivo. Una conclusión que solo existe en la conversación es una conclusión que se evapora al cerrar la terminal.

Sobre qué pedir en esa primera sesión, porque la elección condiciona la impresión que te llevas: **algo real y pequeño de tu proyecto**. Ni «hola» —que no muestra nada— ni «refactoriza el módulo de pagos» —que muestra demasiado sin que puedas juzgarlo.

Lo que mejor funciona el primer día es pedirle que *lea* antes que escriba: «explícame qué hace este módulo y cómo se conecta con el resto». Ahí ves cómo explora, qué archivos elige mirar y si entendió la estructura — que es la información que necesitas para saber cuánto te puedes fiar después.

Dos costumbres que se cogen el primer día y ahorran muchas horas después.

Trabaja sobre git limpio. Antes de pedir algo grande, comprueba que no tienes cambios sin guardar. Es lo que convierte cualquier destrozo en un `git checkout .` en

vez de en una tarde de reconstrucción, y es la razón por la que se puede dejar a un agente tocar archivos sin ansiedad.

Pide una cosa cada vez. Una petición con tres encargos dentro produce tres resultados a medias. Y cuando falla, no sabes cuál de los tres lo rompió — que es el coste real, más que el resultado en sí.

LA SESIÓN QUE SE ALARGA HASTA QUE EMPEORA

Una sesión larga acumula contexto: cada archivo leído y cada salida de comando se quedan. Llega un punto en que el agente responde peor, y no es que se canse — es que lo relevante está enterrado bajo cosas de hace media hora.

La señal es reconocible: empieza a repetir cosas que ya hizo, o a proponer cambios en archivos que dejaron de importar hace rato.

La costumbre que lo evita: **una sesión por tarea**. Cuando cambies de asunto, sal y vuelve a entrar. Lo que necesitas conservar entre sesiones no se guarda en la conversación — se escribe en el proyecto, que es de lo que trata el módulo siguiente.

QUÉ ESTÁ PASANDO MIENTRAS PIENSA

La primera sesión desconcierta porque el agente hace cosas que no le pediste: lee archivos, mira la estructura del proyecto, ejecuta un comando. No se está desviando — está construyendo el contexto que necesita.

Esa es la diferencia con un chat: un chat solo sabe lo que le pegas, y este va a buscarlo. Por eso la primera petición de una sesión tarda más que las siguientes, y por eso conviene arrancarlo **desde la carpeta del proyecto** y no desde tu carpeta personal.

Si lo arrancas en el sitio equivocado, va a leer archivos que no tienen que ver con nada y a gastar contexto en ello.

Una costumbre que vale más que cualquier atajo: **termina cada petición diciendo cómo se sabe que salió bien**. «Añade el campo y comprueba que las pruebas pasan» produce un resultado distinto a «añade el campo», porque la primera le da una condición de parada que puede verificar y la segunda le deja decidir a él cuándo terminó.

Una sesión se abre escribiendo `claude` dentro de la carpeta de un proyecto. A partir de ahí escribes en español. No hay sintaxis que aprender.

Lo que sí conviene saber desde el principio son cuatro atajos que se escriben dentro de la sesión y empiezan por barra:

ESCRIBES	QUÉ HACE	CUÁNDO LO USAS
<code>/help</code>	Lista todo lo que puedes escribir	La primera vez, y cada vez que actualices
<code>/clear</code>	Olvida la conversación y empieza limpio	Al cambiar de tarea: evita que mezcle contextos
<code>/compact</code>	Resume lo hablado para liberar espacio	Cuando la conversación es larga y quieres seguir
<code>/exit</code>	Cierra la sesión	Al terminar

EL ERROR DE NO USAR /CLEAR

La conversación entera viaja con cada mensaje. Si llevas una hora hablando de la pantalla de login y le pides algo del sistema de pagos, arrastra todo lo anterior: responde más lento, mezcla contextos y se le cuelan detalles del tema viejo.

Una tarea, una sesión. Cuando cambies de asunto, `/clear`. Es gratis y arregla la mayoría de las quejas de «se puso tonto».

EJERCICIO 0.2 · 8 minutos · abrir, preguntar y salir

OBJETIVO

Tener una conversación completa con Claude Code sin pedirle todavía que cambie nada.

PASOS

1. Entra en la carpeta de un proyecto tuyo y ábrelo:

● TERMINAL

```
$ cd /ruta/a/tu-proyecto  
$ claude
```

2. Pídele algo que no toque archivos. Por ejemplo: «explícame en tres frases qué hace este proyecto y por dónde empieza a ejecutarse».
3. Escribe `/help` y lee la lista una vez. No hace falta memorizarla.
4. Sal con `/exit` y marca el checkpoint:

● TERMINAL

```
$ ./verificar.sh 0.4 --hecho
```

QUÉ DEBERÍAS VER

Una respuesta que menciona archivos reales de tu proyecto, no genéricos. Si te habla de «un archivo de configuración» sin nombrarlo, es que no leyó nada: pídele que lo mire.

SI ALGO FALLA

Pregunta si confía en la carpeta → es lo normal la primera vez en cada proyecto. Es la protección de la que trata el módulo 2.

Responde en inglés → escríbele en español y seguirá en español. Si quieres fijarlo para siempre, se hace en el módulo 1.

Abriste una sesión y saliste con /exit **MANUAL**

➤ Documentación oficial: Claude Code (<https://docs.claude.com/en/docs/claude-code/overview>)

➤ Referencia de la línea de comandos (<https://docs.claude.com/en/docs/claude-code/cli-reference>)

MÓDULO 1

La memoria del proyecto

CLAUDE.md: lo que no quieres volver a explicar nunca más.

Por qué cada sesión empieza de cero

Y una comprobación que conviene hacerse cada pocas semanas: abrir el `CLAUDE.md` y preguntarse, regla por regla, si sigue siendo verdad. Los comandos cambian, las carpetas se reorganizan, y un archivo de reglas viejo enseña cosas falsas con toda la autoridad de estar escrito.

Este es el módulo con más retorno del curso: es el que hace que las sesiones siguientes empiecen sabiendo, en vez de preguntando. Si solo pudieras hacer uno, sería este.

Hay un patrón que conviene evitar: escribir el `CLAUDE.md` de golpe, un domingo, imaginando todo lo que el agente debería saber. Sale largo, teórico, y con reglas que nadie ha comprobado.

El que funciona crece al revés: empieza casi vacío y gana una línea cada vez que corriges algo. A las dos semanas tiene diez reglas, todas nacidas de un problema real, y todas comprobadas al menos una vez.

Sobre el tamaño: el archivo se lee en cada sesión, así que ocupa contexto siempre. Trescientas líneas de reglas son trescientas líneas menos para tu código, cada vez.

La proporción sana es que quepa en una pantalla larga. Si necesitas más, lo que sobra suele ser un procedimiento —y eso va en una skill, que solo se carga cuando hace falta — o documentación, que va en su archivo y se referencia.

Vale la pena poner un ejemplo de la diferencia. Sin reglas, la primera petición de cada día incluye implícitamente: cómo se corren las pruebas, qué gestor de paquetes usáis, qué carpeta no se toca. Se lo dices tú o lo deduce — y deducirlo cuesta lecturas de archivo y a veces se equivoca.

Con reglas, esa información ya está antes de que abras la boca. La primera petición del día pasa de ser un briefing a ser una petición, y esa diferencia se nota más cuanto más veces al día abres una sesión.

Un detalle práctico: puedes tener `CLAUDE.md` en subcarpetas, no solo en la raíz. Un monorepo con reglas distintas por paquete se maneja así, y el agente lee el que corresponde a donde está trabajando.

Hay un efecto secundario de escribir el `CLAUDE.md` del que casi nadie avisa: **mejora el proyecto**, no solo la relación con el agente.

Poner por escrito qué comandos se usan, qué no se toca y cuándo algo está terminado saca a la luz las cosas que nadie había escrito nunca y que cada persona del equipo tenía en la cabeza de forma ligeramente distinta. Ese archivo acaba siendo lo primero que lee alguien nuevo — agente o humano.

Por eso conviene revisarlo en los pull requests como cualquier otro archivo. Un `CLAUDE.md` que se queda viejo enseña cosas falsas con mucha autoridad.

TRES SITIOS DONDE PONER MEMORIA, Y CUÁL USAR

El proyecto (`CLAUDE.md` en la raíz) — lo que vale para cualquiera que trabaje aquí. Se versiona con el código y lo hereda quien clone. Es el que importa.

Tú (`~/ .claude/CLAUDE.md`) — tus preferencias personales, en todos tus proyectos. Cómo quieres que te hable, qué idioma, qué nivel de detalle.

Local (sin versionar) — lo que solo aplica a tu copia: rutas de tu máquina, credenciales de un entorno de pruebas tuyo.

El error frecuente es meter en el del proyecto cosas que son tuyas —«explícame las cosas en detalle»— y acabar imponiéndole tu estilo a todo el equipo. Si la frase empieza por «yo prefiero», va en el tuyo, no en el del proyecto.

El curso trae un `CLAUDE.md` de partida con los apartados que de verdad se usan:

● TERMINAL

```
$ cp plantilla/CLAUDE.md /ruta/a/tu-proyecto/
```

EL ARCHIVO QUE CRECE HASTA QUE DEJA DE LEERSE

El fallo de este módulo no es no tener `CLAUDE.md` : es tener uno de doscientas líneas que nadie ha comprobado.

Cada regla que añades diluye a las demás. Un archivo con cinco reglas que se cumplen vale más que uno con cuarenta declaradas, porque el agente reparte su atención y tú pierdes la capacidad de saber cuáles funcionan.

La prueba está en el ejercicio de este módulo, y es la parte más importante del curso: **escribe una regla, pide algo que la active, y comprueba que la cumplió sin recordársela**. Si no la cumplió, esa regla está escrita pero no vive.

Método para saber qué escribir: durante una semana, cada vez que corrijas al agente, anota la corrección en una línea. Al final tendrás entre cinco y diez, y son exactamente las que tu proyecto necesita — no las que recomienda un artículo genérico.

Claude Code no recuerda la sesión de ayer. Es una decisión de diseño, no un defecto: así no arrastra suposiciones viejas ni datos de otro proyecto. Pero tiene un coste evidente — vuelves a explicar lo mismo cada mañana.

La solución es un archivo de texto llamado `CLAUDE.md` . Se lee solo al abrir la sesión, y lo que pongas ahí es como si se lo hubieras dicho tú antes de empezar.

DOS ALTURAS, DOS PROPÓSITOS

`~/ .claude/CLAUDE.md` — tus preferencias personales, para **todos** tus proyectos. Idioma, estilo, qué no quieres que haga nunca.

`CLAUDE.md` en la raíz del proyecto — las reglas de **ese** proyecto: cómo se ejecutan las pruebas, qué carpetas no se tocan, qué librería se usa. Este se comparte con el equipo por git.

Los dos se leen y se suman. Lo personal no pisa lo del proyecto: conviven.

La diferencia entre un archivo que funciona y uno que se ignora es la **concreción**. «Escribe código limpio» no cambia nada. «Las pruebas se ejecutan con `npm test`, nunca con `jest` directamente» sí.

REGLA QUE NO SIRVE	REGLA QUE SÍ
Sigue las buenas prácticas	Los nombres de función en inglés y en <code>camelCase</code>
Escribe tests	Toda función nueva en <code>src/core/</code> lleva su prueba al lado
No rompas nada	No modifiques <code>migrations/</code> : son históricas
Usa buen estilo de commits	Commits en inglés, formato <code>tipo: descripción</code>

EJERCICIO 1.1 · 12 minutos · escribir tus dos memorias

OBJETIVO

Un archivo personal con tus preferencias y otro con las reglas de tu proyecto, comprobando que los lee.

PASOS

1. Dile al verificador dónde practicas:

● TERMINAL

```
$ export CURSO_PROYECTO=/ruta/a/tu-proyecto
```

2. Crea o abre tu archivo personal. Escribe tres o cuatro líneas: en qué idioma quieres las respuestas, qué nunca debe hacer sin preguntar, cómo prefieres que te explique las cosas.

● TERMINAL

```
$ claude  
> /memory
```

3. En la raíz de tu proyecto, crea `CLAUDE.md` con cuatro o cinco reglas concretas de ese proyecto.
4. Comprueba que los dos existen y tienen contenido:

● TERMINAL

```
$ ./verificar.sh -m 1
```

5. Abre una sesión nueva y pídele algo que toque una de tus reglas, **sin mencionarla**. Si dijiste que las pruebas van con `npm test`, pídele que añada una prueba y mira qué comando usa.
6. Cuando confirmes que la respetó sola:

● TERMINAL

```
$ ./verificar.sh 1.3 --hecho
```

QUÉ DEBERÍAS VER

El verificador te dice cuántos caracteres útiles tiene cada archivo. Y en la sesión, el agente debería seguir tu regla sin que la repitas. Ese es el momento en que el archivo deja de ser teoría.

SI ALGO FALLA

El verificador dice que tiene pocos caracteres útiles → descuenta el encabezado de metadatos, así que cuenta solo lo que escribiste de verdad.

Escribiste el archivo pero lo ignora → comprueba que está en la **raíz** del proyecto donde abres la sesión, no en una subcarpeta. Dentro de la sesión, `/memory` te muestra qué archivos cargó.

Le pusiste demasiadas reglas → treinta reglas se diluyen. Empieza con cinco que de verdad te importen y añade cuando notes que falta una.

Tienes un CLAUDE.md global con tus preferencias AUTO

Tu proyecto tiene su propio CLAUDE.md AUTO

Comprobaste que respeta una regla sin recordársela MANUAL

➤ Documentación oficial: memoria y CLAUDE.md (<https://docs.claude.com/en/docs/claude-code/memory>)

MÓDULO 2

Permisos

Decidir de antemano qué puede hacer sin preguntarte, y qué no debe hacer nunca.

El equilibrio que hay que ajustar una vez

Cuando el agente pida un permiso que no tienes configurado, la respuesta correcta casi nunca es dárselo para siempre en ese momento. Concédelo para esa vez, termina lo que estabas haciendo, y decide después con calma si merece entrar en la lista.

Media hora bien invertida aquí es lo que después permite dejarlo trabajar sin vigilarlo. Los permisos no son burocracia de seguridad: son la condición para que el agente pueda comprobar su propio trabajo sin interrumpirte cada tres pasos.

Un detalle que evita sustos: los permisos se aplican también a lo que el agente ejecuta *dentro* de un script. Permitir un comando que a su vez ejecuta cualquier cosa es abrir la puerta entera sin darse cuenta.

Por eso las listas de permitidos que funcionan son específicas — `npm test`, `git diff` — y no genéricas. Un permiso amplio para «cualquier npm» incluye `npm run` de un script que hace lo que sea.

Sobre saltarse los permisos del todo: existe la opción y no conviene salvo en un entorno desechable — un contenedor, una máquina virtual, un repositorio de pruebas.

El riesgo real no es que el agente decida hacer daño: es que un comando razonable con una ruta mal deducida haga algo irreversible en tu máquina. En un contenedor eso cuesta reconstruirlo; en tu portátil, cuesta lo que costara.

Una consecuencia de los permisos que se descubre tarde: **son lo que hace viable el modo sin conversación**. Si tu configuración depende de que tú apruebes cosas a mano, el agente no puede correr en integración continua ni en un cron.

Por eso conviene resolver los permisos bien una vez, aunque cueste media hora: es el trabajo que después habilita los módulos 9 y 10, no un trámite de seguridad.

Los permisos admiten patrones, no solo nombres de herramienta. `Bash(git diff:*)` permite cualquier `git diff` pero no cualquier `git`, y esa granularidad es la que hace viable tener una lista de permitidos cómoda sin abrir la mano del todo.

Los permisos tienen tres sitios donde vivir, y el orden importa: la configuración del proyecto (`.claude/settings.json` , versionada, la que comparte el equipo), la tuya local para ese proyecto (sin versionar), y la global de tu usuario.

Lo que va en cada sitio: **las denegaciones importantes van en la del proyecto**, para que apliquen a todo el mundo. Los permisos que te dan comodidad a ti —dejarle correr un comando concreto de tu entorno— van en la local, porque son tuyos y puede que a otro no le sirvan.

LOS DOS EXTREMOS QUE SE PAGAN IGUAL

Aprobarlo todo a mano convierte cada tarea en una sucesión de preguntas y anula la ventaja de que el agente pueda comprobar su trabajo. A la media hora estás aceptando sin leer, que es lo peor de los dos mundos: la fricción sin la seguridad.

Dar permiso a todo funciona hasta el día que no. Y el problema no es imaginar al agente haciendo algo destructivo a propósito: es que un comando razonable con un argumento mal deducido borra lo que no debía.

El punto medio es aburrido y funciona: **todo lo que lee, permitido; lo que escribe, permitido dentro del proyecto; lo que no tiene vuelta atrás, denegado siempre**. Esa tercera lista es corta — `git push` , borrados recursivos, credenciales— y es la que de verdad te protege.

El curso trae una configuración de permisos de partida, pensada para ser restrictiva en lo que importa y cómoda en lo demás:

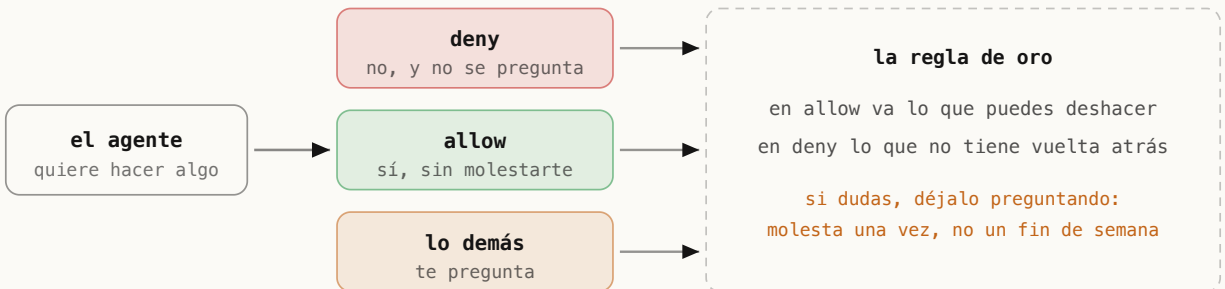
● TERMINAL

```
$ cp -r plantilla/.claude /ruta/a/tu-proyecto/
```

Léela antes de copiarla. La lista de `deny` es la parte que conviene entender: bloquea leer `.env` y archivos de credenciales, y bloquea `git push` y `rm -rf` . No porque el agente sea malicioso, sino porque esas cuatro son las que no tienen vuelta atrás.

Recién instalado, Claude Code pregunta por casi todo. Eso es correcto al principio y cansa a la media hora: acabas dando a «sí» sin leer, que es exactamente el hábito que la protección quería evitar.

El arreglo no es desactivar las preguntas, es **decidir de antemano** qué te da igual y qué no. Eso vive en `~/.claude/settings.json`, en un bloque llamado `permissions`.



Cada acción pasa por los mismos tres filtros. Lo que no está en ninguna lista te pregunta.

CÓMO SE ESCRIBE UNA REGLA

Una regla nombra la herramienta y, entre paréntesis, a qué se aplica: `Bash(git status)`, `Bash(npm test)`, `Read(src/**)`. El asterisco vale como comodín, así que `Bash(git *)` cubre todos los comandos de git.

Van en tres listas: `allow` (hazlo sin preguntar), `deny` (no lo hagas nunca) y `ask` (pregunta siempre, aunque encaje en allow).

LO QUE NUNCA DEBERÍA ESTAR EN ALLOW

La regla práctica: **en allow va solo lo que puedes deshacer**. Leer archivos, correr pruebas, ver el estado de git.

Fuera van los comandos sin vuelta atrás: `rm -rf` , `git push --force` , borrar ramas, cualquier cosa contra producción. Esos van en `deny` , y si necesitas uno alguna vez, lo escribes tú a mano en ese momento.

La asimetría importa: una autorización de más te puede costar un fin de semana; una pregunta de más te cuesta dos segundos.

EJERCICIO 2.1 · 12 minutos · escribir tus permisos

OBJETIVO

Dejar configurado qué puede hacer sin preguntarte y qué no debe hacer nunca, y comprobar que la lista de prohibidos funciona.

PASOS

1. Mira cómo está hoy tu configuración:

● TERMINAL

```
$ ./verificar.sh -m 2
```

2. Abre los ajustes desde dentro de una sesión — es más seguro que editar el archivo a mano, porque valida lo que escribes:

● TERMINAL

```
$ claude  
> /permissions
```

3. Añade a `allow` tres cosas que hagas a diario y puedas deshacer. Buenas candidatas: `Bash(git status)`, `Bash(git diff)`, `Bash(npm test)`.
4. Añade a `deny` lo que no quieres ni por accidente: `Bash(rm -rf *)`, `Bash(git push --force*)`.
5. Comprueba que quedó guardado:

● TERMINAL

```
$ ./verificar.sh 2.2
```

6. Pídele en la sesión que haga algo de tu lista de prohibidos. Debe negarse. Cuando lo veas:

● TERMINAL

```
$ ./verificar.sh 2.3 --hecho
```

QUÉ DEBERÍAS VER

El verificador cuenta cuántas reglas tienes en cada lista, por ejemplo `permissions: 32 en allow, 15 en deny`. Y al pedirle algo prohibido, el agente lo rechaza sin ofrecerte confirmarlo: eso es lo que distingue `deny` de una pregunta.

SI ALGO FALLA

Editaste el JSON a mano y dejó de funcionar → suele ser una coma de más al final de una lista. El verificador te dice si el archivo dejó de ser JSON válido.

Pusiste una regla en allow y sigue preguntando → el patrón tiene que coincidir con el comando completo. `Bash(npm test)` no cubre `npm test -- --watch`; para eso, `Bash(npm test*)`.

Te tiente desactivar todos los permisos → existe la opción y tiene su sitio: máquinas desechables sin acceso a tus datos. En tu equipo de trabajo, no.

settings.json existe y es JSON válido **AUTO**

Definiste reglas de permisos **AUTO**

Viste una petición de permiso y la denegaste **MANUAL**

➤ Documentación oficial: permisos y control de acceso (<https://docs.claude.com/en/docs/claude-code/iam>)

➤ Referencia de settings.json (<https://docs.claude.com/en/docs/claude-code/settings>)

Comandos propios

Convertir lo que escribes cada semana en algo que se llama con una barra.

Un archivo de texto se vuelve un comando

Los comandos viven en archivos de texto, así que se versionan, se revisan en un pull request y se corrigen como cualquier otra cosa del proyecto. No hay ninguna interfaz que aprender.

Merece la pena ver la diferencia entre pedir algo suelto y tener el comando. Sin comando escribes cada vez: «revisa el diff, reporta solo lo que causaría un fallo real, no comentarios estilo, y dime para cada hallazgo qué entrada lo rompe». Con comando escribes `/revisar`.

Lo que se ahorra no es teclear: es que la parte larga —el criterio— se aplique **igual todas las veces**. Cuando lo escribes a mano, la versión de las siete de la tarde es más corta que la de las nueve de la mañana, y los resultados se resienten sin que sepas por qué.

Y si un comando tuyo deja de usarse, bórralo. La lista de comandos es un espacio compartido con los del sistema, y llenarla de cosas muertas hace que la que sí usas cueste encontrarla.

A partir de aquí el curso deja de ser configuración básica y pasa a ser extensión: cosas que añades cuando algo se repite lo suficiente como para que valga la pena escribirlo una vez.

Sobre cuántos tener: pocos y usados. Cinco comandos que invocas a diario valen más que veinte que documentan todo lo que tu equipo sabe hacer, porque los veinte no caben en la cabeza de nadie.

Y si acabas con muchos, el prefijo común es lo que los hace navegables. La lista se agrupa sola y encuentras el que buscas sin recordar su nombre exacto.

Los comandos pueden ejecutar cosas antes de que el agente lea el resto: por ejemplo, meter la salida de `git diff` directamente en el contexto sin que tenga que pedirla.

Eso ahorra una ronda entera en los comandos que siempre trabajan sobre lo mismo, y es lo que separa un comando que se siente instantáneo de uno que empieza explorando cada vez.

Hay comandos que vienen ya con la herramienta — `/model` , `/clear` , `/help` — y los tuyos conviven con ellos. Escribe `/` y sale la lista entera, la suya y la tuya, que es la forma más rápida de recordar qué tienes.

Un consejo de nomenclatura: si tu equipo va a tener varios, ponles un prefijo común. `/pr-revisar` , `/pr-preparar` se agrupan solos en la lista; `/revisar` y `/preparar` quedan sueltos entre los del sistema.

CÓMO SE ESCRIBE UN COMANDO QUE SE ACABA USANDO

La mayoría de los comandos que se escriben el primer día se abandonan el tercero. Los que sobreviven comparten tres cosas:

Resuelven algo que haces varias veces por semana. Si es mensual, no vas a recordar que existe.

Llevan dentro un criterio, no solo pasos. «Revisa el diff» lo pides sin comando; «revisa el diff con *nuestro* criterio de qué bloquea» necesita estar escrito.

Tienen un nombre que adivinarías. `/revisar` se encuentra; `/rev-pr-check` hay que recordarlo, y no lo vas a recordar.

Y una prueba rápida antes de darlo por bueno: pásaselo a alguien del equipo sin explicárselo. Si tiene que preguntarte qué hace, el que necesita trabajo es el nombre o la descripción, no el contenido.

Los comandos admiten argumentos, y ahí es donde dejan de ser una nota fija. Escribes `$ARGUMENTS` en el archivo y lo que teclees después del comando se sustituye ahí.

Eso convierte `/revisar` en `/revisar src/pagos` , o un comando de despliegue en uno que recibe el entorno. Un buen comando suele tener una o dos variables y un

criterio largo — al revés de como se escriben la primera vez, con muchas variables y ningún criterio.

Dónde se guardan decide quién los tiene. En `.claude/commands/` del proyecto, se versionan y los hereda el equipo entero. En `~/claude/commands/`, son tuyos en todos tus proyectos.

Como regla: si el comando encapsula un criterio *del proyecto* —qué es bloqueante aquí, cómo se despliega esto— va en el proyecto. Si encapsula una manía tuya, va en tu carpeta personal.

El curso trae un comando de ejemplo, `/revisar`, en `plantilla/.claude/commands/revisar.md`. Míralo antes de escribir el tuyo: es corto a propósito y muestra la parte que casi todo el mundo omite — **decir también qué NO reportar**.

UN COMANDO NO ES UN ALIAS

El error de este módulo es escribir comandos que solo ahorran teclear: `/test` que ejecuta `npm test`. Para eso ya está la terminal, y el agente en medio solo añade latencia.

Un comando gana su sitio cuando encapsula **un criterio**, no una acción.

`/revisar` vale porque lleva dentro qué es bloqueante y qué no en tu proyecto — un juicio que si no está escrito, cada sesión lo improvisa distinto.

Prueba de fuego: si tu comando se puede sustituir por un alias de shell, no debería ser un comando.

Cuando llevas un tiempo notas que repites peticiones casi idénticas: «revisa este cambio buscando fallos de seguridad», «escribe el mensaje de commit siguiendo nuestro formato». Cambiar tres palabras cada vez es trabajo perdido.

Un **comando propio** es un archivo de texto con esa petición dentro. Si lo guardas como `revisar.md`, dentro de la sesión lo llamas escribiendo `/revisar`.

DÓNDE SE GUARDAN

`~/ .claude/commands/` — disponibles en todos tus proyectos.

`.claude/commands/` dentro del proyecto — solo ahí, y se comparten con el equipo por git.

El nombre del archivo es el nombre del comando. `desplegar.md` se convierte en `/desplegar`.

Dentro del archivo escribes la petición como se la dirías tú. Si quieres que reciba datos al llamarlo, usas `$ARGUMENTS`, que se sustituye por lo que escribas después del comando.

EJERCICIO 3.1 · 12 minutos · crear tu primer comando

OBJETIVO

Convertir una petición que repites en un comando que se llama con una barra.

PASOS

1. Piensa en algo que le pidas al menos una vez por semana. Si no se te ocurre, empieza por este: revisar un cambio antes de subirlo.

2. Crea la carpeta y el archivo:

● TERMINAL

```
$ mkdir -p ~/.claude/commands  
$ nano ~/.claude/commands/revisar.md
```

3. Escribe dentro la petición completa, como se la dirías tú. Por ejemplo: «Revisa los cambios sin subir de este repositorio. Busca errores de lógica, datos sensibles escritos a mano y funciones que no se usen. Dime solo lo que haya que arreglar, ordenado por gravedad.»

4. Comprueba que quedó registrado:

● TERMINAL

```
$ ./verificar.sh 3.1
```

5. Abre una sesión, escribe `/revisar` y confírmalo:

● TERMINAL

```
$ ./verificar.sh 3.2 --hecho
```

QUÉ DEBERÍAS VER

El verificador lista tus comandos con su nombre, por ejemplo `/build-check`, `/commit-push-pr`. Dentro de la sesión, `/` los muestra junto a los que vienen

de serie.

SI ALGO FALLA

No aparece en la lista de la sesión → si la sesión ya estaba abierta cuando creaste el archivo, ciérrala y ábrela otra vez.

El nombre tiene espacios o acentos → usa solo letras, números y guiones.

`revisar-pr.md` sí, `revisar PR.md` no.

Hace algo distinto cada vez → la petición es demasiado vaga. Un buen comando dice qué mirar, qué ignorar y en qué formato responder.

Creaste un comando propio con / **AUTO**

Lo ejecutaste dentro de una sesión **MANUAL**

➤ Documentación oficial: comandos con barra (<https://docs.claude.com/en/docs/claude-code/slash-commands>)

MÓDULO 4

Skills

Procedimientos completos que se cargan solos cuando hacen falta.

La diferencia entre una regla y un procedimiento

Escribe la descripción pensando en cómo lo pediría alguien que no sabe que la skill existe. Esa es la única frase que el agente va a leer para decidir.

Un buen indicador de que algo debería ser skill: cuando te descubres explicándole a alguien —o al agente— los mismos siete pasos por tercera vez. Esa tercera vez es la señal.

Las skills son la pieza que menos gente usa y la que más cambia el trabajo en equipo, porque convierten «cómo se hace esto aquí» en algo que se ejecuta igual sin que

nadie tenga que acordarse.

Cómo saber si una skill se está usando: pídele algo que debería activarla y mira si la menciona. Si no aparece, el problema está en la descripción, no en los pasos.

Es la comprobación equivalente a la del módulo 1 con las reglas, y por la misma razón: una skill que nunca se invoca está escrita pero no vive.

Una skill puede vivir en el proyecto o en tu carpeta personal, igual que los comandos. La decisión es la misma: si el procedimiento es de este proyecto, va versionado con él; si es tu forma de trabajar en todos, va en la tuya.

En equipos, las skills del proyecto son la forma más efectiva de que un procedimiento se haga igual sin escribir un documento que nadie lee. El agente sí lo lee, y lo lee cada vez.

Hay skills que vienen dadas —para trabajar con documentos de oficina, por ejemplo— y skills que escribes tú. Las primeras sirven para ver el formato antes de escribir la tuya: son un buen modelo de cómo se redacta una descripción que el agente sabe cuándo usar.

El tamaño razonable de una skill propia es una pantalla de pasos más los archivos de apoyo que haga falta. Si el `SKILL.md` se te va a cinco pantallas, probablemente estás describiendo tres procedimientos y conviene separarlos.

Una forma de decidir rápido si algo debería ser skill: **¿lo explicarías igual a alguien nuevo cada vez?** Si la respuesta es sí, es un procedimiento y va en una skill. Si cambia según el caso, es criterio y va en las reglas.

Ejemplos de procedimiento: cómo se prepara una versión, cómo se revisa un PR, cómo se añade un idioma nuevo. Ejemplos de criterio: qué estilo usamos, qué no se toca, cuándo algo está terminado.

Una skill puede traer más archivos además del `SKILL.md`: guiones, plantillas, documentos de referencia. El agente los lee solo cuando los necesita, no de entrada.

Eso resuelve el problema de siempre con la documentación larga: no puedes meter treinta páginas en las reglas del proyecto porque se comerían el contexto de cada sesión. Con una skill, las treinta páginas están disponibles y solo entran cuando la tarea las pide.

LA DESCRIPCIÓN ES LO ÚNICO QUE SE LEE PARA DECIDIR

Una skill que nadie invoca es una skill que no existe, y la causa casi siempre es la misma: la descripción no dice **cuándo** usarla.

«Ayuda con revisiones de código» no le sirve al agente para decidir nada. «Úsalo cuando se pida revisar un PR, revisar cambios antes de mergearear, o comprobar si una rama está lista» sí, porque enumera las formas en que la gente pide realmente esa tarea.

Escribe la descripción pensando en las palabras que usarías tú, no en el nombre técnico de lo que hace.

El curso trae una skill completa en `plantilla/.claude/skills/revisar-pr/`, con su cabecera, sus pasos y —lo que casi nadie escribe— su criterio de qué es bloqueante.

CUÁNDO ES REGLA, CUÁNDO COMANDO Y CUÁNDO SKILL

Las tres cosas se confunden constantemente porque las tres son texto que influye en el agente. Se distinguen por **cuándo entran en juego**:

Regla (`CLAUDE.md`) — vale siempre, en cada sesión, sin que nadie la invoque. «El SQL va parametrizado.»

Comando (`/nombre`) — lo invocas tú, explícitamente, cuando quieres.
«Revísame el diff.»

Skill — la invoca el agente solo, cuando la tarea encaja con su descripción. De ahí que la descripción sea la parte más importante del archivo: es lo único que el agente lee para decidir si le sirve.

Si escribes una skill con una descripción vaga, no se va a usar nunca y vas a concluir que las skills no funcionan.

Una regla de `CLAUDE.md` dice *cómo comportarse* y está siempre presente. Un **skill** enseña *cómo hacer algo concreto*: un procedimiento con sus pasos, y si hace falta con sus archivos de apoyo.

POR QUÉ NO ES LO MISMO QUE UNA REGLA

Las reglas están siempre cargadas y ocupan sitio en cada conversación. Un skill **se carga solo cuando el agente decide que viene al caso**, o cuando lo llamas escribiendo `/nombre-del-skill`.

Esa diferencia es la que permite tener cincuenta procedimientos guardados sin que ninguno estorbe cuando no toca.

Un skill es una carpeta con un archivo `SKILL.md` dentro. Ese archivo lleva un encabezado con dos datos que importan mucho:

- **name** — el nombre con el que lo llamas.

- **description** — cuándo debe usarse. Esta es la parte crítica: es lo único que el agente lee para decidir si cargarlo. Una descripción vaga hace que el skill no se active nunca, o que se active siempre.

LA DESCRIPCIÓN DECIDE SI EL SKILL EXISTE

«Ayuda con despliegues» no le dice nada al agente. «Usar cuando se pida desplegar a producción, hacer rollback, o revisar por qué falló un despliegue» sí: nombra las situaciones concretas en las que aplica.

Si escribes un skill y nunca se activa, el problema casi siempre está en esa línea, no en el contenido.

EJERCICIO 4.1 · 15 minutos · escribir un skill para tu proyecto

OBJETIVO

Guardar un procedimiento de tu proyecto como skill y comprobar que queda disponible.

PASOS

1. Mira qué skills tienes ya:

● TERMINAL

```
$ ./verificar.sh 4.1
```

2. Elige un procedimiento de varios pasos que hagas de vez en cuando y siempre tengas que recordar: preparar una versión, cargar datos de prueba, revisar antes de un despliegue.

3. Crea la carpeta dentro de tu proyecto:

● TERMINAL

```
$ mkdir -p $CURSO_PROYECTO/.claude/skills/mi-procedimiento  
$ nano $CURSO_PROYECTO/.claude/skills/mi-procedimiento/SKILL.md
```

4. Empieza el archivo con el encabezado entre tres guiones, y debajo los pasos:

● ARCHIVO · SKILL.MD

```
---  
name: mi-procedimiento  
description: Usar cuando se pida preparar una versión nueva  
---  
  
1. Comprobar que las pruebas pasan  
2. Actualizar el número de versión  
3. ...
```


5. Comprueba que quedó bien formado:

● TERMINAL

```
$ ./verificar.sh 4.2
```

QUÉ DEBERÍAS VER

El verificador confirma el skill y su nombre. En una sesión abierta en ese proyecto, escribir `/` debería mostrarlo entre las opciones.

SI ALGO FALLA

El verificador no lo encuentra → tiene que ser una carpeta con un archivo llamado exactamente `SKILL.md` en mayúsculas, dentro de `.claude/skills/`.

Existe pero nunca se activa solo → reescribe la `description` nombrando las situaciones concretas. Mientras tanto puedes llamarlo a mano con `/mi-procedimiento`.

Quieres que esté en todos tus proyectos → entonces va en `~/.claude/skills/` en vez de dentro del proyecto.

Tienes skills disponibles `AUTO`

Escribiste un skill para tu proyecto `AUTO`

➤ Documentación oficial: skills (<https://docs.claude.com/en/docs/claude-code/skills>)

MÓDULO 5

Subagentes

Delegar una parte del trabajo a otro agente con su propio contexto.

Para qué sirve tener más de uno

Y cuando delegues, pon en el encargo todo lo que el subagente necesita: no ve tu conversación, así que las referencias a «lo que acabamos de hacer» no significan nada para él.

Y una advertencia sobre el entusiasmo inicial: montar cinco subagentes especializados antes de necesitarlos produce una configuración bonita que nadie invoca. Empieza por el verificador, que es el único que casi siempre compensa, y añade otro cuando un trabajo concreto lo pida.

Los subagentes resuelven dos problemas distintos que conviene no mezclar: **paralelizar** trabajo independiente, y **separar** a quien juzga de quien hace. El segundo es el que más vale y el que menos se usa.

Un patrón que rinde y casi nadie monta: un subagente que solo **investiga** y no toca nada. Le pides que averigüe cómo funciona una parte del sistema, vuelve con un resumen, y el agente principal trabaja con eso sin haber gastado su contexto en la exploración.

Es la forma más limpia de trabajar en un proyecto grande: la exploración, que es lo que más contexto consume, ocurre fuera.

Sobre el coste: cada subagente consume su propio presupuesto. Tres en paralelo es tres veces el gasto de uno, y a veces vale la pena y a veces no.

La regla práctica: si el trabajo que le delegas te habría costado más de diez minutos a ti, casi siempre compensa. Si son tres lecturas de archivo, hacerlo directo sale más barato y más rápido.

Conviene saber cuándo *no* usar subagentes, porque es más frecuente que lo contrario. Si la tarea es secuencial —cada paso necesita el resultado del anterior— repartirla no acelera nada y sí multiplica el contexto que hay que reconstruir.

El caso claro es el opuesto: cinco archivos que se pueden revisar sin depender unos de otros, o una investigación donde tres preguntas distintas se pueden responder en

paralelo. Ahí sí hay tiempo real que ganar.

Un uso concreto que vale más que la explicación general: **dos subagentes con criterios distintos sobre el mismo trabajo**. Uno mira correctitud, otro mira seguridad. Cada uno encuentra cosas que el otro no ve, porque están buscando cosas distintas.

Es más útil que pedirle a uno solo «revísalo todo», que produce listas largas y superficiales. Un lente concreto por revisor da hallazgos concretos.

Los subagentes se guardan como archivos en `.claude/agents/`, con su descripción y —esto es lo importante— **su propia lista de herramientas**.

Restringirlas no es solo seguridad: es enfoque. Un verificador con permiso para editar acabará arreglando lo que encuentre, y entonces deja de ser un verificador. El del curso solo puede leer y ejecutar, y por eso su informe sirve.

Un detalle que sorprende la primera vez: **los subagentes no heredan tu conversación**. Arrancan limpios y solo saben lo que les pongas en el encargo.

Eso es una virtud —por eso el verificador funciona— y una trampa: si le pides a un subagente que «arregle lo que acabamos de ver», no sabe de qué hablas. Hay que decírselo entero.

El curso trae un subagente de ejemplo en

`plantilla/.claude/agents/verificador.md`, y es el que más se usa de los tres artefactos: comprueba si un trabajo terminado cumple lo que se pidió, sin haberlo escrito él.

POR QUÉ UN VERIFICADOR APARTE FUNCIONA MEJOR

Un agente que acaba de escribir doscientas líneas es mal juez de esas doscientas líneas. No por falta de capacidad: por contexto. Tiene delante todo el razonamiento que le llevó a escribirlas, y ese razonamiento es persuasivo — incluso cuando el resultado no funciona.

Un subagente arranca con contexto limpio. Solo ve lo que le pasas y lo que puede comprobar por su cuenta, así que no tiene nada que defender.

Es el mismo principio que sostiene los bucles autónomos: **quien hace el trabajo no decide si está terminado**. Aquí se aplica en pequeño, dentro de una sesión, y es lo que más reduce las veces que el agente dice «listo» con las pruebas en rojo.

EL COSTE DE DELEGAR

Cada subagente vuelve a construir su contexto desde cero: relee archivos, vuelve a explorar, y después te devuelve un informe que el agente principal tiene que leer. Eso multiplica el gasto y el tiempo.

Delega cuando el trabajo sea **independiente y de tamaño real** —una investigación amplia, varios archivos que se pueden mirar en paralelo, una verificación con criterio propio—. No delegues lo que tú mismo resolverías con tres lecturas de archivo: sale más caro que hacerlo.

Hay tareas que ensucian la conversación: buscar en cuarenta archivos dónde se usa una función, revisar un cambio largo, leer registros. Todo eso entra en el contexto, ocupa sitio y te deja una sesión llena de ruido cuando lo que querías era el resumen.

Un **subagente** hace ese trabajo en una conversación aparte y te devuelve solo la conclusión. La búsqueda de cuarenta archivos ocurre en su contexto, no en el tuyo.

CUÁNDO DELEGAR Y CUÁNDO NO

Delega cuando el trabajo genera mucho texto intermedio que no necesitas: rastrear algo por el proyecto, revisar con un criterio concreto, resumir registros.

No delegues lo que requiere ir y venir contigo. El subagente no te puede preguntar a mitad: recibe una tarea y devuelve un resultado.

Se definen como archivos en `~/claude/agents/` o en `.claude/agents/` del proyecto. Cada uno lleva su descripción, sus instrucciones y, opcionalmente, qué herramientas puede usar — un revisor que solo lee no debería poder escribir.

EJERCICIO 5.1 · 14 minutos · crear un revisor

OBJETIVO

Definir un subagente que revise cambios con tu criterio, y delegarle una revisión real.

PASOS

1. Mira si ya tienes alguno:

● TERMINAL

```
$ ./verificar.sh 5.1
```

2. Créalo con el asistente que trae la propia sesión, que es más rápido que escribir el archivo a mano:

● TERMINAL

```
$ claude  
> /agents
```

3. Descríbelo por lo que debe buscar, no por lo que debe ser. En vez de «eres un revisor experto», algo como: «revisa los cambios sin subir buscando datos sensibles escritos a mano, funciones sin usar y errores de lógica; responde con una lista ordenada por gravedad».
4. Dale solo las herramientas de lectura si su trabajo es revisar. Que no pueda escribir es parte del diseño.
5. Comprueba que quedó guardado y después délegale una revisión de verdad:

● TERMINAL

```
$ ./verificar.sh 5.1
```

6. Cuando veas su informe:

● TERMINAL

```
$ ./verificar.sh 5.2 --hecho
```

QUÉ DEBERÍAS VER

El verificador lista tus subagentes por nombre. Al delegarle una tarea, la sesión principal te muestra que arrancó y después te entrega su informe, sin todo el texto intermedio que generó por el camino.

SI ALGO FALLA

El informe es genérico → la descripción es vaga. Un subagente con instrucciones amplias devuelve resultados amplios.

Tarda mucho → es lo esperado si le diste medio proyecto. Acota: «revisa solo los cambios sin subir», no «revisa el repositorio».

Modificó archivos y no querías → no le quitaste las herramientas de escritura. Edítalo y déjale solo lectura.

Definiste al menos un subagente **AUTO**

Delegaste una tarea y viste su informe **MANUAL**

➤ Documentación oficial: subagentes (<https://docs.claude.com/en/docs/claude-code/sub-agents>)

MÓDULO 6

Hooks

Ejecutar algo tuyo, siempre, cuando pasa un evento. Sin depender de que el agente se acuerde.

La diferencia entre pedirlo y garantizarlo

Prueba siempre el hook a mano antes de configurarlo. Un fallo aquí no se nota una vez: se nota en todas las sesiones, hasta que lo encuentras.

Antes de escribir un hook, pregúntate si el incumplimiento ocasional te cuesta algo de verdad. Si la respuesta es que no, una regla basta y es más barata de mantener. Los hooks se reservan para lo que no puede fallar ni una vez.

Este módulo es corto y su idea es la más portátil del curso: sirve para agentes, para procesos de equipo y para casi cualquier sitio donde alguien confunda escribir una norma con hacerla cumplir.

Sobre por dónde empezar: el hook más rentable del primer día es formatear al guardar. Es inofensivo, se nota enseguida, y te enseña el mecanismo sin riesgo de bloquear nada.

Los que bloquean —los que devuelven error— déjalos para cuando tengas claro qué quieres impedir. Un hook bloqueante mal calibrado se nota mucho y en todas partes.

Los hooks se configuran en el mismo archivo que los permisos, y como ellos pueden ser del proyecto o tuyos. Los del proyecto se versionan, así que todo el equipo tiene la misma garantía.

Ese es el argumento más fuerte para usarlos en equipo: una regla escrita se cumple según quién y cuándo; un hook versionado se cumple igual para todos, incluida la persona que se incorporó ayer.

Un uso de los hooks que casi nadie considera y que resuelve un problema real: **avisar**. Un hook que suena o notifica cuando el agente termina te deja lanzarle algo largo y volver a lo tuyo sin estar mirando la terminal.

Suena menor y cambia bastante el uso diario: la alternativa es quedarse esperando, que es exactamente lo que estas herramientas deberían evitarte.

Un hook recibe información sobre lo que está pasando —qué herramienta, qué archivos— en variables de entorno, y puede decidir en función de eso. Y si termina con

un código de error, bloquea la operación.

Ahí está toda la potencia y todo el peligro: un hook mal escrito que devuelve error por accidente bloquea cosas legítimas en todas tus sesiones. Por eso los que solo avisan o formatean deben terminar siempre en éxito, y solo los que de verdad tienen que bloquear devuelven error.

Los hooks se enganchan en varios momentos, y elegir bien cuál cambia lo que puedes hacer:

MOMENTO	CUÁNDO SE EJECUTA	PARA QUÉ SIRVE
PreToolUse	Antes de que la herramienta actúe	Bloquear lo que no debe pasar
PostToolUse	Después, con el resultado	Formatear, normalizar, comprobar
Stop	Cuando el agente cree que terminó	Comprobar que de verdad terminó
SessionStart	Al abrir la sesión	Cargar estado, avisar de algo pendiente

El de `Stop` es el menos usado y el más interesante: es donde se pone la comprobación que impide que «listo» signifique «listo para mí».

UN HOOK MAL ESCRITO ROMPE TODAS LAS SESIONES

Los hooks se ejecutan en cada operación que coincida con su patrón, así que un error ahí no falla una vez: falla siempre, en todo lo que hagas.

Dos precauciones que ahorran una tarde. **Termina en** `|| true` los hooks que no deben bloquear —formatear, avisar— para que un fallo del formateador no impida guardar archivos. Y **pruébalo primero a mano** en la terminal con un archivo de ejemplo, antes de ponerlo en la configuración.

El hook del curso lleva las dos cosas: `--no-install` para no instalar nada por sorpresa, y `|| true` para que su fallo no se lleve por delante la edición.

LA FRONTERA ENTRE PROMPT Y CÓDIGO

Este módulo tiene la idea más transferible del curso, y vale mucho más allá de Claude Code.

Un prompt es una **instrucción probabilística**: se cumple casi siempre. Un hook es **código**: se cumple siempre, porque no depende de que nadie lo interprete.

La consecuencia práctica es una regla de decisión: **si el incumplimiento ocasional te cuesta caro, no lo pidas —garantízalo**. Formatear el código al guardar puede ser una regla; bloquear un despliegue a producción hasta que pasen las pruebas tiene que ser un hook.

Casi todo el mundo descubre esta frontera al revés: escribiendo instrucciones cada vez más enfáticas —«CRÍTICO», «SIEMPRE», «NUNCA»— hasta que entiende que el problema no era el énfasis, sino la capa.

Puedes escribir en `CLAUDE.md` «ejecuta el formateador después de editar». A veces lo hará y a veces no: es una instrucción, y las instrucciones se interpretan.

Un **hook** no se interpreta. Es un comando tuyo que el programa ejecuta **siempre** que ocurre un evento, decida el agente lo que decida. Es la diferencia entre pedir algo y

garantizarlo.

LOS EVENTOS QUE MÁS SE USAN

PreToolUse — justo antes de que use una herramienta. Puede **bloquearla**: es el sitio para prohibir de verdad algo peligroso.

PostToolUse — después. El sitio del formateador y del linter.

UserPromptSubmit — cuando envías un mensaje. Sirve para inyectar contexto automáticamente.

Stop — cuando el agente termina de responder. Para avisos y comprobaciones finales.

Se configuran en `settings.json`, dentro del bloque `hooks`. Cada evento lleva una lista de comandos, y esos comandos reciben por la entrada estándar un JSON con lo que está pasando: qué herramienta, con qué argumentos, en qué archivo.

UN HOOK LENTO SE NOTA EN CADA MENSAJE

Los hooks corren de forma síncrona: mientras tu comando no termine, la sesión espera. Un hook que tarda tres segundos convierte cada edición en tres segundos de pausa.

Que sean rápidos y silenciosos. Si necesitas algo pesado, que el hook lo lance en segundo plano y devuelva el control enseguida.

EJERCICIO 6.1 · 15 minutos · un hook que se note

OBJETIVO

Configurar un hook que haga algo visible después de cada edición, y verlo dispararse.

PASOS

1. Mira si ya tienes hooks configurados:

● TERMINAL

```
$ ./verificar.sh 6.1
```

2. Configúralo desde la sesión, que valide el formato:

● TERMINAL

```
$ claude  
> /hooks
```

3. Elige el evento `PostToolUse` y como comando algo inofensivo y visible. En un Mac, esto avisa cada vez que edita un archivo:

● TERMINAL

```
$ osascript -e 'display notification "Archivo editado" with title "Claude C
```

4. Si prefieres algo útil desde el primer día, pon el formateador de tu proyecto en su lugar.

5. Comprueba que quedó guardado:

● TERMINAL

```
$ ./verificar.sh 6.1
```

6. Pídele en la sesión que edite cualquier archivo. Cuando veas el hook dispararse:

● TERMINAL

```
$ ./verificar.sh 6.2 --hecho
```

QUÉ DEBERÍAS VER

El verificador te dice cuántos eventos tienen hook, por ejemplo `11` evento(s) con hook: `UserPromptSubmit`, `PreToolUse`, `PostToolUse...` . Y al editar un archivo, tu comando se ejecuta sin que nadie se lo pida.

SI ALGO FALLA

No se dispara → los hooks se cargan al abrir la sesión. Si la tenías abierta, ciérrala y vuelve a entrar.

La sesión se puso lenta → tu comando tarda demasiado. Pruébalo suelto en la terminal y cronométralo.

El hook falla y bloquea todo → un hook que devuelve error en `PreToolUse` impide la acción, que a veces es lo que quieres y a veces no. Si te bloqueaste, edita `settings.json` a mano y quita el bloque.

Configuraste un hook en settings.json **AUTO**

Viste el hook dispararse **MANUAL**

➤ Documentación oficial: hooks (<https://docs.claude.com/en/docs/claude-code/hooks>)

MÓDULO 7

MCP

Darle acceso a cosas que están fuera de tu carpeta.

Qué problema resuelve

Y revisa cada cierto tiempo qué tienes conectado. Los servidores MCP se acumulan como los plugins, ocupando contexto en cada sesión aunque hayas dejado de usarlos.

Y una comprobación de seguridad antes de conectar nada: mira qué permisos tiene la credencial que le vas a dar. En la mayoría de los casos basta con solo lectura, y esa decisión de treinta segundos limita el alcance de cualquier error posterior.

MCP es la capa que conecta al agente con lo que no está en tu disco. Es potente y es la que más fácil se sobre-configura, así que conviene entrar con la pregunta puesta: ¿qué copio y pego todos los días?

Comprobar qué tienes conectado es una orden, y conviene mirarlo de vez en cuando: los servidores se acumulan igual que los plugins, y cada uno sigue ocupando su sitio aunque hayas dejado de usarlo.

Una revisión cada pocos meses, quitando lo que no invocas, devuelve contexto a lo que sí haces.

Vale la pena distinguir MCP de las herramientas que Claude Code ya trae. Leer archivos, ejecutar comandos o buscar en la web son tuyas y están siempre. MCP es para lo que *no* es tu máquina: un sistema externo con su propia autenticación.

La confusión frecuente es montar un servidor MCP para algo que ya se puede hacer con un comando. Si `psql` está instalado y el agente puede ejecutarlo, quizá no necesites el servidor — necesitas el permiso.

Una recomendación práctica sobre por dónde empezar con MCP: por el servidor que te ahorre el copiar y pegar que más repites. Si cada día pegas resultados de consultas, empieza por la base de datos; si cada día pegas descripciones de incidencias, por el gestor de incidencias.

Empezar por «el que parece más potente» lleva casi siempre a tener conectado algo que no usas, con el coste de contexto que eso arrastra.

Un ejemplo hace más que la definición. Sin MCP, para que el agente sepa qué hay en tu base de datos tienes que ejecutar la consulta tú y pegarle el resultado. Con MCP, le pides «mira cuántos pedidos quedaron en estado pendiente esta semana» y lo consulta él.

La diferencia no es la comodidad de no copiar y pegar: es que puede **iterar**. Ve el resultado, se da cuenta de que la consulta no era la correcta, y prueba otra. Eso con copiar y pegar cuesta tres rondas tuyas.

Conviene saber distinguir los dos sitios donde se configura un servidor MCP, porque decide quién lo tiene. En el proyecto (`.mcp.json` , versionado) lo hereda el equipo; en tu configuración de usuario, es solo tuyo.

Y una regla sobre credenciales que no admite excepción: **nunca en el archivo versionado**. Van en variables de entorno que el archivo referencia. Un `.mcp.json` con una clave dentro es una clave publicada en cuanto alguien clone el repositorio.

QUÉ ES MCP EN UNA FRASE, Y QUÉ NO ES

Es un protocolo para que el agente hable con sistemas que no son archivos: tu base de datos, tu gestor de incidencias, un navegador. Lo que gana es que esas capacidades aparecen como herramientas tuyas, así que puede **consultar** en vez de pedirte que le pegues el resultado.

Lo que no es: una forma de darle más inteligencia, ni un sustituto de tener buenas reglas. Un agente con diez servidores MCP y sin `CLAUDE.md` sigue sin saber cómo se ejecutan tus pruebas.

Y una advertencia de seguridad que conviene tomar en serio: un servidor MCP puede leer y escribir en el sistema al que conecta. Conectar uno es darle esa capacidad al agente, así que revisa qué permisos tiene la credencial que le das — casi siempre puede ser de solo lectura.

CADA SERVIDOR MCP TIENE UN COSTE QUE NO SE VE

La tentación con MCP es conectar todo lo que existe: base de datos, gestor de incidencias, calendario, navegador. Y hay un coste que no aparece hasta que ya duele.

Cada servidor conectado **mete sus herramientas en el contexto de cada sesión**, la use o no. Con cinco servidores medianos, una parte notable del presupuesto de contexto se va en describir capacidades que no vas a usar hoy — y ese espacio sale del que tenía para tu código.

Conecta lo que uses de verdad esta semana. Es fácil añadir uno más cuando haga falta y es muy difícil notar que sobra.

Claude Code lee tu proyecto y ejecuta comandos. Pero tu trabajo no vive solo ahí: hay tickets, una base de datos, la documentación de una librería, el panel donde despliegas.

MCP es el enchufe estándar para eso. Un servidor MCP le da al agente un conjunto de herramientas nuevas — consultar tus tickets, leer una tabla — sin que tengas que copiar y pegar información.

VOCABULARIO MÍNIMO

Las siglas son de Model Context Protocol. Lo importante es que es **un estándar**: el mismo servidor sirve para Claude Code, para Cursor y para otras herramientas. Lo que configures aquí no se pierde si cambias.

Se añaden con `claude mcp add`. Hay dos formas habituales: un servicio por internet (transporte `http`) o un programa que corre en tu máquina.

CLAUDE MCP LIST PUEDE MOSTRAR TUS CREDENCIALES

Muchos servidores se configuran con una clave dentro del propio comando. Cuando después ejecutas `claude mcp list`, esa clave **aparece en pantalla en texto plano**.

No es un fallo, pero tiene dos consecuencias prácticas: no ejecutes ese comando mientras compartes pantalla, y no pegues su salida en un ticket o en un chat sin revisarla. Si necesitas enseñar tu configuración, tapa las claves.

EJERCICIO 7.1 · 12 minutos · conectar tu primer servidor

OBJETIVO

Añadir un servidor MCP y comprobar que el agente puede usar sus herramientas.

PASOS

1. Mira qué tienes conectado hoy. Ojo con la advertencia de arriba si no estás solo:

● TERMINAL

```
$ claude mcp list
```

2. Añade uno. Si no tienes ninguno en mente, uno de documentación es buen primer caso, porque se nota enseguida:

● TERMINAL

```
$ claude mcp add --transport http <nombre> <url-del-servidor>
```

3. Comprueba que aparece y que conecta:

● TERMINAL

```
$ ./verificar.sh 7.1
```

4. Abre una sesión y pídele algo que solo pueda responder usando ese servidor. Si conectaste uno de documentación, pídele la firma exacta de una función de esa librería.

QUÉ DEBERÍAS VER

El verificador cuenta cuántos servidores tienes y cuántos conectan, por ejemplo **8 servidor(es) MCP configurado(s), 3 conectado(s)**. Que un servidor esté configurado y no conectado es normal: suele faltar autenticarse.

SI ALGO FALLA

Dice «Needs authentication» → el servidor está bien puesto pero falta iniciar sesión. Desde una sesión, `/mcp` te lleva al proceso.

Configurado pero el agente no usa sus herramientas → pídeselo explícitamente la primera vez, nombrando el servidor. Si sigue sin usarlo, comprueba con `/mcp` que la sesión lo ve.

Aviso de que los conectores están desactivados → suele salir cuando hay una `ANTHROPIC_API_KEY` definida, que tiene prioridad sobre tu sesión de claude.ai. Es informativo, no un error.

Tienes al menos un servidor MCP configurado **AUTO**

➤ Documentación oficial: MCP en Claude Code (<https://docs.claude.com/en/docs/claude-code/mcp>)

MÓDULO 8

Plugins

Instalar de una vez lo que otros ya configuraron.

Un paquete con todo dentro

Y una última advertencia práctica antes de entrar: instalar un plugin ajeno es ejecutar configuración escrita por otra persona en tu máquina, hooks incluidos. Míralo con el mismo criterio con que mirarías un script que te pasan por correo — no con desconfianza, pero sí abriéndolo antes.

Una nota sobre el ecosistema: la mayoría de los plugins públicos que vas a encontrar son colecciones de comandos para flujos concretos — trabajar con un framework, seguir una metodología, integrarse con un servicio. Suelen ser cortos y se leen en cinco minutos, que es exactamente lo que conviene hacer antes de instalar cualquiera.

Los plugins son el último módulo de extensión y el más opcional del curso. Si trabajas solo, puedes saltártelo sin perder nada: tu carpeta `.claude/` versionada con el proyecto hace lo mismo. Léelo cuando aparezca la segunda persona o el segundo repositorio.

Dicho eso, entender qué son ayuda aunque no escribas ninguno, porque explica cómo se distribuyen las configuraciones que te vas a encontrar por ahí.

QUÉ CABE DENTRO DE UN PLUGIN

Un plugin puede llevar las cuatro cosas que has ido montando en los módulos anteriores, y esa es exactamente su gracia:

Comandos — los `/nombre` del módulo 3, disponibles para quien lo instale.

Skills — los procedimientos del módulo 4, con sus archivos de apoyo.

Subagentes — los del módulo 5, con sus herramientas ya restringidas.

Hooks — los del módulo 6, que es la parte que conviene leer antes de instalar nada ajeno, porque son comandos que se van a ejecutar en tu máquina.

Lo que no lleva son las reglas del proyecto: esas son de cada proyecto y viven en su `CLAUDE.md`.

La decisión de empaquetar suele llegar sola: cuando te descubres copiando la misma carpeta `.claude/` al tercer repositorio, ya tienes el argumento hecho.

Si acabas escribiendo uno para tu equipo, empieza por lo que ya usáis y funciona. Un plugin que empaqueta tres cosas probadas se adopta; uno que propone un flujo nuevo que nadie ha probado se instala y se olvida.

Los plugins son distribución, no capacidad. Todo lo que traen funciona igual suelto; lo que aportan es que otra persona lo tenga idéntico con una orden.

Un plugin bien hecho es también documentación. Al leerlo se ve cómo alguien resolvió un flujo entero: qué puso en reglas, qué en comandos, qué garantizó con hooks.

Aunque no lo instales, leer dos o tres enseña más sobre cómo se combinan las piezas que cualquier explicación — incluida esta.

Sobre de dónde salen: hay un mercado de plugins públicos y también se pueden distribuir por un repositorio propio, que es lo habitual dentro de una empresa.

Para uso interno, un repositorio de git con el plugin dentro basta. No hace falta publicar nada ni pasar por ningún registro, y así el contenido no sale de la organización.

Un plugin puede traer también servidores MCP configurados, no solo comandos y skills. Eso lo convierte en la forma más completa de repartir una configuración: alguien instala el plugin y tiene lo mismo que tú, incluidas las conexiones.

Con la salvedad de siempre: las credenciales no van dentro. El plugin trae la configuración; cada persona pone su clave en su entorno.

EL PLUGIN QUE INSTALAS Y NUNCA USAS

El patrón es reconocible: instalas tres plugins interesantes, cada uno trae cuatro comandos, y a la semana no recuerdas ninguno de los doce. El coste no es cero — sus hooks siguen corriendo y sus skills siguen ocupando sitio en cada sesión.

Instala uno cada vez y dale una semana. Si en esa semana no lo invocaste, desinstálalo: no era para ti, o no era para ahora.

Lo mismo vale para lo que construyas tú. Un plugin propio con dos cosas que el equipo usa a diario vale más que uno con veinte que documentan todo lo que sabéis hacer.

Vale la pena mirar plugins existentes antes de escribir el tuyo, aunque solo sea para ver cómo están montados. Muchos son un buen ejemplo de cómo se combinan comandos, skills y hooks para un flujo concreto.

Y si acabas escribiendo uno para tu equipo, la parte que decide si se usa no es el contenido: es el README. Un plugin sin instrucciones de qué hace y cuándo conviene usarlo se instala una vez y se olvida.

Y una nota sobre instalar plugins ajenos: un plugin puede traer hooks, y un hook es un comando que se ejecuta en tu máquina. Léelo antes, como leerías un script que te pasan por correo. La mayoría son inofensivos y esa comprobación cuesta dos minutos.

CUÁNDO EMPAQUETAR Y CUÁNDO NO

Un plugin junta comandos, skills, subagentes y hooks en algo instalable. Su valor no es técnico —todo eso funciona igual suelto— sino de **distribución**: que otra persona lo tenga exactamente igual que tú, con una orden.

Por eso solo compensa cuando hay alguien más. Si trabajas solo, tus archivos en `.claude/` versionados con el proyecto hacen lo mismo con menos ceremonia.

El caso donde sí gana con claridad: un equipo con varios repositorios donde quieres que la revisión de código se haga igual en todos. Ahí el plugin es el mecanismo, y la alternativa es copiar y pegar hasta que las copias diverjan.

Los cuatro módulos anteriores son cosas que escribes tú: comandos, skills, subagentes, hooks. Un **plugin** es un paquete que trae varias de esas piezas juntas, ya configuradas, listo para instalar.

Se instalan desde un marketplace, que no es más que un repositorio con una lista. Puedes usar los públicos o montar el de tu equipo, que es la forma práctica de que ocho personas trabajen con la misma configuración.

CUÁNDO INSTALAR Y CUÁNDO ESCRIBIR EL TUYO

Instala cuando el problema no es tuyo: revisar pull requests, trabajar con un proveedor concreto, un flujo estándar de la industria.

Escribe el tuyo cuando el procedimiento es de tu casa. Un plugin ajeno que hace el 70% de lo que necesitas suele costar más de adaptar que escribir el tuyo desde cero.

EJERCICIO 8.1 · 10 minutos · instalar un plugin

OBJETIVO

Añadir un marketplace, instalar un plugin y comprobar que sus piezas quedaron disponibles.

PASOS

1. Mira qué tienes instalado:

● TERMINAL

```
$ claude plugin list
```

2. Desde una sesión, el catálogo se explora con un comando:

● TERMINAL

```
$ claude  
> /plugin
```

3. Instala uno que te sirva de verdad. Si dudas, uno de revisión de código es el que antes se nota.

4. Comprueba que quedó registrado:

● TERMINAL

```
$ ./verificar.sh 8.1
```

5. Escribe **/** en la sesión: los comandos que trae el plugin aparecen con su nombre delante.

QUÉ DEBERÍAS VER

El verificador cuenta los plugins instalados y nombra algunos. En la sesión, sus comandos salen prefijados con el nombre del plugin, para que no choquen con los tuyos.

Aparece como `disabled` → instalado pero desactivado. Se activa desde `/plugin`, o en `settings.json` dentro de `enabledPlugins`.

Sus comandos no aparecen → cierra y abre la sesión. Los plugins se cargan al arrancar.

Instalaste varios y ahora hay ruido → desinstala lo que no uses. Cada plugin añade comandos y skills que compiten por la atención del agente.

Instalaste un plugin desde un marketplace AUTO

➤ Documentación oficial: plugins (<https://docs.claude.com/en/docs/claude-code/plugins>)

MÓDULO 9

Sin sesión interactiva

Lamarlo desde un script, un cron o tu integración continua.

El mismo agente, sin conversación

Este módulo es donde el curso deja de ser sobre tu forma de trabajar y pasa a ser sobre la del proyecto: lo que montes aquí corre igual esté quien esté delante.

Un ejemplo concreto de por dónde empezar: un comando que, cada mañana, lea los commits del día anterior y escriba un resumen en un archivo. No decide nada, no toca código, y te enseña el mecanismo completo con riesgo cero.

Desde ahí, el salto a algo que sí decide —bloquear un merge, abrir una incidencia— es pequeño, y llegas con la infraestructura ya probada.

Antes de automatizar algo, hazlo a mano varias veces. Lo que automatizas sin haberlo hecho antes acaba siendo una automatización de un proceso que no era el correcto, y esos son los más difíciles de desmontar.

Aquí es donde todo lo anterior se paga. Las reglas, los permisos y los verificadores que montaste son exactamente lo que hace posible que el agente trabaje sin nadie delante — sin ellos, este módulo no se sostiene.

Sobre el coste en automatización: aquí es donde se descontrola si nadie lo mira. Un agente en cada pull request de un repositorio activo son muchas ejecuciones al día.

Empieza por lo programado —una vez al día, una vez por semana— antes que por cada evento. El cron te deja ver el gasto real antes de multiplicarlo por el número de PR.

Sobre dónde se ejecuta esto de verdad en la práctica: integración continua, ganchos de git, tareas programadas. Los tres sitios donde ya tienes automatización y donde el agente encaja sin inventar infraestructura nueva.

Y en los tres aplica lo mismo que ya sabes del módulo de permisos: lo que no esté permitido de antemano va a bloquear la ejecución, porque no hay nadie para autorizarlo.

Un uso que rinde desde el primer día sin montar nada: **pasarle texto por una tubería**. Los registros de un fallo, la salida de un comando que no entiendes, un diff largo. El agente lo lee y te contesta, sin abrir sesión.

Es la puerta de entrada más natural a este modo, y la que hace evidente el salto siguiente: si eso funciona a mano, funciona igual dentro de un script.

LO QUE SALE MAL CUANDO NADIE MIRA

Tres fallos que en una sesión interactiva se corrigen solos y aquí no:

La tarea se queda a medias y termina igual. Sin condición de aceptación escrita, el agente decide él cuándo terminó, y con frecuencia decide pronto. Ponla siempre en el encargo.

Se queda esperando. Si algo le pide confirmación —un permiso que no diste, un comando interactivo— no hay nadie para contestar. Los permisos tienen que estar resueltos *antes*.

Cuesta más de lo previsto. Una tarea mal acotada puede iterar mucho. Conviene poner un límite de tiempo o de intentos en el propio encargo.

Los tres se resuelven en el mismo sitio: escribiendo el encargo completo antes de lanzarlo, en vez de irlo aclarando sobre la marcha.

La forma de salida importa más de lo que parece. En modo no interactivo puedes pedir el resultado como JSON, y eso convierte al agente en algo que otro programa puede consumir: un paso más de una tubería, no un final.

Es la diferencia entre «me escribe un resumen que leo yo» y «me devuelve un veredicto que decide si el pipeline sigue». La segunda es la que hace que esto encaje en integración continua sin adaptaciones.

LO QUE CAMBIA CUANDO NO HAY NADIE MIRANDO

Sin conversación no hay a quién preguntarle, y eso cambia cómo se escribe la petición. Todo lo que en una sesión resolverías con un «no, así no» tiene que estar en el encargo desde el principio: el objetivo, las restricciones y **cómo se sabe que terminó**.

Es también donde los permisos dejan de ser una comodidad y pasan a ser lo único que te protege: no vas a estar ahí para decir que no.

La forma más útil de este modo no es «que haga la tarea entera solo», sino encadenarlo: un comando que revisa cada pull request, otro que resume los cambios de la semana. Tareas acotadas, repetitivas y con criterio escrito.

Todo lo anterior ocurre en una sesión donde tú escribes y él responde. Pero el mismo programa se puede llamar **una sola vez, con una instrucción, y recoger la respuesta**. Eso abre la puerta a automatizar.

Se hace con `-p` (de *print*): ejecuta la petición, imprime el resultado y termina.

● TERMINAL

```
$ claude -p "resume en tres líneas qué cambió en el último commit"
```

PARA QUÉ SE USA DE VERDAD

Un resumen automático de los cambios de la semana. Una primera revisión de cada pull request antes de que la vea una persona. Un script que traduce archivos nuevos. Un aviso cuando algo raro aparece en los registros.

La clave es que **encaja donde ya tienes automatización**: un cron, un GitHub Action, un script que ya corres.

Cuando lo llamas desde un programa conviene pedir la respuesta en un formato que se pueda leer con código, no en prosa. Para eso está `--output-format`.

AUTOMATIZAR HEREDA TUS PERMISOS

Un script no tiene a nadie delante para responder a una pregunta de permiso. Es tentador desactivar todas las comprobaciones para que no se quede colgado, y ahí está el peligro: un agente sin supervisión y sin límites, corriendo solo.

Lo correcto es al revés: **dale una lista corta de herramientas permitidas** con `--allowedTools`, la mínima para esa tarea. Si el script solo lee y resume, no necesita poder escribir.

EJERCICIO 9.1 · 15 minutos · tu primer script

OBJETIVO

Escribir un script que llame a Claude Code sin abrir sesión y devuelva algo útil.

PASOS

1. Confirma que tu versión ofrece el modo no interactivo:

● TERMINAL

```
$ ./verificar.sh 9.1
```

2. Pruébalo suelto, para ver la forma de la respuesta:

● TERMINAL

```
$ claude -p "lista los tres archivos más grandes de este proyecto"
```

3. Crea un script en tu proyecto que haga algo que repitas. Este resume los cambios sin subir:

● ARCHIVO · RESUMEN.SH

```
#!/usr/bin/env bash
set -euo pipefail

git diff --stat | claude -p \
  "Resume estos cambios en tres viñetas para un mensaje de commit" \
  --allowedTools "Read"
```

4. Hazlo ejecutable y Pruébalo:

● TERMINAL

```
$ chmod +x resumen.sh
$ ./resumen.sh
```

5. Comprueba el checkpoint:

● TERMINAL

```
$ ./verificar.sh 9.2
```

QUÉ DEBERÍAS VER

El script devuelve texto y termina, sin abrir ninguna conversación. El verificador confirma que existe un script tuyo que usa `claude -p`.

SI ALGO FALLA

Se queda esperando → algo pidió un permiso que nadie puede conceder. Acota con `--allowedTools` a lo que de verdad necesita.

Responde con prosa cuando querías datos → pídele el formato explícitamente en la instrucción, o usa `--output-format json` y léelo con código.

Funciona a mano pero falla en el cron → un cron no hereda tu entorno. Comprueba que la ruta de `claude` y las variables de autenticación estén disponibles ahí.

Tu versión ofrece el modo no interactivo AUTO

Escribiste un script que llama a claude -p AUTO

➤ Referencia de la línea de comandos (<https://docs.claude.com/en/docs/claude-code/cli-reference>)

➤ Documentación oficial: modo headless (<https://docs.claude.com/en/docs/claude-code/headless>)

MÓDULO 10

El día a día con git

Dónde encaja el agente en el ciclo de trabajo real, y dónde no.

Lo que conviene delegarle y lo que no

Con esto termina el curso. Lo que queda montado no está en tu máquina: está en tu repositorio, versionado, y lo hereda quien clone el proyecto.

Y si te llevas una sola costumbre del curso entero, que sea esta: **trabaja siempre sobre git limpio y en una rama**. Es lo que convierte cualquier experimento en algo reversible, y lo que te deja darle permisos amplios sin que eso sea una imprudencia.

Último módulo, y el más práctico: qué le das y qué te quedas en el trabajo diario con git, que es donde acaba pasando todo lo demás.

Y un cierre sobre la relación con git en general: todo lo que hace seguro trabajar con un agente ya existía antes de que hubiera agentes. Ramas, commits pequeños, poder deshacer.

Quien ya trabajaba así apenas cambia nada y gana mucho. Quien no, va a notar que la primera mejora útil de este curso no fue configurar al agente: fue ordenar cómo trabaja con su propio repositorio.

Una última costumbre que rinde más de lo que parece: pedirle que **lea el diff antes de que lo leas tú** y te cuente qué cambió y qué le preocupa.

No sustituye tu revisión: la ordena. Llegas al diff sabiendo dónde mirar, y eso convierte una revisión de veinte minutos en una de cinco sin perder lo que importa.

Con esto se cierra el curso. Tienes las reglas del proyecto escritas, los permisos ajustados, comandos y skills para lo que repites, un verificador que comprueba lo que tú no vas a comprobar, hooks para lo que no puede fallar, y el agente corriendo sin conversación donde hace falta.

Todo eso vive en tu repositorio, se versiona con el código y lo hereda quien clone. Que es la diferencia entre haber configurado tu máquina y haber configurado el proyecto.

RAMAS: LA RED QUE HACE QUE TODO LO ANTERIOR SEA SEGURO

Una costumbre que cambia el nivel de riesgo de todo el curso: **trabaja en una rama cuando le pidas algo grande.**

No es por ceremonia de proceso. Es que una rama convierte cualquier resultado —bueno, malo o incomprensible— en algo que se descarta con un comando. Sin ella, un cambio de quince archivos que no acaba de funcionar es una tarde de deshacer a mano.

Con esa red puesta, puedes darle permisos más amplios y dejarle iterar más, que es cuando estas herramientas rinden de verdad. La gente que las usa con miedo suele ser la que no tiene dónde caerse.

Una forma concreta de aprovecharlo con git que casi nadie usa: **pedirle que investigue el historial.** «¿En qué commit se introdujo este comportamiento?» es una pregunta que a mano cuesta veinte minutos de `git log` y `git bisect`, y que él contesta en dos porque puede ejecutar los dos comandos e interpretar la salida.

Ese tipo de tarea —buscar en un historial, cruzar información de varios commits, encontrar cuándo cambió algo— es donde la diferencia con hacerlo tú es mayor. Escribir código es donde más se le mira; investigar es donde más ahorra.

LO QUE NO CONVIENE DELEGARLE NUNCA

Hay tres cosas donde el agente es rápido y tú eres responsable, y la velocidad no compensa:

Escribir el mensaje de commit sin leerlo. Es el registro que va a leer alguien dentro de un año, tú incluido. Que lo redacte, sí; que se publique sin que lo mires, no.

Resolver conflictos de fusión. Un conflicto es dos intenciones que chocan, y la información para decidir cuál gana casi nunca está en el código.

Cualquier cosa contra la rama principal. `push` , `rebase` , `reset --hard` . Por eso están en la lista de denegados de la plantilla.

Lo que sí conviene delegar entero: leer el diff y contarte qué cambió, preparar el borrador del mensaje, encontrar en qué commit se introdujo un fallo. Ahí es donde de verdad ahorra.

Claude Code es bueno con git porque git se maneja por comandos y devuelve texto: puede ver el estado, leer un diff y proponer un mensaje de commit que describa lo que de verdad cambió.

Pero hay una asimetría que conviene respetar desde el principio.

DELÉGALE	HAZLO TÚ
Redactar el mensaje del commit leyendo el diff	Decidir qué entra en el commit
Resumir qué cambió en una rama	Fusionar a la rama principal
Encontrar el commit que introdujo un fallo	Reescribir historia que ya compartiste
Preparar la descripción de un pull request	Aprobar y publicar

LA REGLA QUE RESUME LA TABLA

Delega lo que **se puede deshacer o solo produce texto**. Quédate con lo que **cambia lo que otros ven**: publicar, fusionar, reescribir historia compartida.

No es desconfianza: es que el coste del error es asimétrico. Un mensaje de commit malo se corrige; un force-push sobre la rama del equipo arruina la mañana de varias personas.

El módulo 2 es lo que hace esto automático. Si `Bash(git push --force*)` está en `deny`, la regla deja de depender de tu memoria.

EJERCICIO 10.1 · 12 minutos · un commit escrito por él, aprobado por ti

OBJETIVO

Dejar que prepare un commit completo a partir de tus cambios, revisarlo y decidir tú.

PASOS

1. Comprueba que tu proyecto de práctica es un repositorio git:

● TERMINAL

```
$ ./verificar.sh 10.1
```

2. Haz un cambio pequeño de verdad en tu proyecto. Que sea real: un cambio inventado da un mensaje inventado.
3. Abre la sesión y pídele que lea lo que cambió y prepare el commit. Que no lo suba:

● TERMINAL

```
$ claude  
> mira los cambios sin subir y prepara un commit con un mensaje que explique
```

4. Lee el mensaje que propuso **antes** de aceptar. Si describe el qué («actualiza archivo») en vez del porqué, díselo y que lo reescriba.
5. Cuando lo hayas revisado y decidido tú:

● TERMINAL

```
$ ./verificar.sh 10.2 --hecho
```

QUÉ DEBERÍAS VER

Un mensaje de commit que menciona el motivo del cambio y no solo los archivos tocados. El verificador confirma que el repositorio está listo y te

muestra el último commit.

SI ALGO FALLA

El mensaje describe archivos, no motivos → es lo que pasa cuando solo ve el diff. Dile en qué estabas trabajando y por qué; con contexto, el mensaje mejora mucho.

Metió en el commit cosas que no querías → decidir qué entra sigue siendo tuyo. Prepara tú lo que va y pídele solo el mensaje.

Lo subió sin preguntar → revisa tus permisos: `Bash(git push*)` no debería estar en `allow`.

El proyecto de práctica es un repositorio git **AUTO**

Dejaste que preparara un commit y lo revisaste **MANUAL**

Con esto tienes el ciclo completo: memoria para no repetirte, permisos para no vigilar, comandos y skills para lo que repites, subagentes para lo que ensucia, hooks para lo que no puede fallar, MCP y plugins para llegar más lejos, y el modo no interactivo para lo que ni siquiera quieres mirar.

➤ Todos los cursos, en el orden en que conviene hacerlos (<https://cursos.rizo.ma>)

El kit local vive en `~/GitHub/curso-claude-code`. Corre `./verificar.sh` para la tabla completa, `./verificar.sh -m 6` para un módulo suelto y `./verificar.sh --resumen` para tu progreso. Los checkpoints automáticos no se marcan a mano: se comprueban contra lo que tienes instalado.

Los comandos y mensajes de este curso están tomados de la ayuda del propio programa y de haberlos ejecutado, no de la memoria. La referencia oficial es la [documentación de Claude Code](https://docs.claude.com/en/docs/claude-code/overview) (<https://docs.claude.com/en/docs/claude-code/overview>), enlazada en cada módulo.

Versión 1.0.0 · actualizado el 3 de agosto de 2026



RIZOMA · RIZO.MA